Corporate On-Chain Governance and Community-Governed Funds

Analysis of the Lux DAO Stack

Zach Kelling

Lux Industries

Corporate On-Chain Governance and Community-Governed Funds

Analysis of the Lux DAO Stack

Lux Industries Inc.

Internal architectural analysis

Abstract

This paper analyzes the Lux DAO stack as a foundation for two related but distinct use cases: (i) corporations adopting on-chain wallets and on-chain governance for treasury, capital, and operational decisions; and (ii) communities operating self-governed funds where token-holder voting allocates pooled capital. The stack comprises a modular governance contract family (the Lux Governor Protocol), parent-child hierarchical DAO structures (Fractal), Safe multisig integration, Hats Protocol role hierarchies, ERC-6551 token-bound accounts, ERC-4337 account abstraction with sponsoring paymasters, freeze-and-resume circuit breakers, warrant tracking for equity-like instruments, and a public-sale module for token launches. We map each component to a corporate-governance or community-fund function, identify gaps in the current implementation, and propose extensions tying the stack to the Lux post-quantum patent family for regulated-custody and compliance-gated settlement. The analysis concludes that the stack is production-ready for community-fund use cases today and is one quarter of focused engineering work away from production-ready for regulated corporate governance use cases.

1 Scope and method

We examine `~/work/lux/dao` as it stands at the analysis date. Sources consulted: the architecture and reality-check documents in the repository (`ARCHITECTURE.md`, `STACK_REALITY.md`, `LLM.md`), the deployable contract families under `contracts/contracts/deployables/`, and the singleton contracts under `contracts/contracts/singletons/`. We do not separately audit the contract security properties or implementation correctness; those are out of scope for this paper and tracked under the audits directory.

The analysis proceeds in three parts. Section 2 maps the existing stack to canonical corporate-governance and community-fund functions. Section 3 analyzes the deployment patterns supported by the stack. Section 4 identifies gaps relative to production-grade regulated use cases and proposes extensions integrating the Lux patent family.

2 Architectural map

The stack decomposes into seven functional layers. Each layer addresses an orthogonal concern; together they form a coherent system for on-chain organizational governance.

2.1 Layer 1: Governance core — the Lux Governor Protocol

The governance core is a fork of the Decent / Fractal modular governance system, rebranded as the Lux Governor Protocol and exposed via the contract family at `contracts/contracts/deployables/modules/`.

Contract	Role
`ModuleGovernorV1`	Central governance module managing the proposal lifecycle. Inherits from OpenZeppelin Governor, configured as a Zodiac-compatible module that can be attached to a Safe multisig. Handles propose, vote, queue, execute.
`ModuleFractalV1`	Parent-child DAO relationships. Permits subDAO creation under a parent DAO with configurable autonomy boundaries; the parent retains optional veto or freeze authority over the child.
`StrategyV1`	Voting strategy module: quorum thresholds, voting periods, timelock, proposer-threshold gating. Pluggable so that different proposal classes can use different strategies.

The governor pattern follows OpenZeppelin's Governor v5 design adapted to function as a Zodiac module rather than as a standalone executive. This allows the Safe multisig to remain the on-chain identity of the organization while the governor mediates which proposals are executed. The strict separation of treasury custody (Safe) from governance logic (Governor) is the load-bearing design choice of the entire stack: it permits governance upgrades without treasury migration and permits multiple governance modules to coexist on a single treasury during transitions.

2.2 Layer 2: Voting weight and proposer authorization

Voting weight is decoupled from the governance core via the strategies layer at `contracts/contracts/deployables`

Contract	Role
`VotingWeightERC20V1`	Token-weighted voting using an ERC-20 votes-extended token (delegate semantics from ERC-20Votes). Standard for community-fund and shareholder-style governance.
`VotingWeightERC721V1`	NFT-based voting (one-NFT-one-vote or stake-weighted). Useful for membership-defined organizations where membership tokens are NFTs.
`ProposerAdapterHatsV1`	Hats Protocol integration. Proposal authority is gated by holding a configured "hat" (role-NFT). Enables role-based proposal rights: only the holder of the "Treasurer" hat may propose treasury motions; only the holder of the "Compliance Officer" hat may propose compliance changes.
`vote-trackers/`	Vote-recording adapters supporting timestamp and block-number clock modes (per EIP-6372). Required for cross-chain governance where block numbers diverge.

The Hats Protocol adapter is the most important component for corporate use cases. Corporations need role-based delegation: the board can authorize officers to act on certain matters without

full vote each time. Hats provides this as on-chain role-NFTs whose holders inherit configured authority. Combined with ProposerAdapterHatsV1, the result is a flexible analog to corporate by-law structure: proposals are routed by role, and revoking a role (burning the hat) cleanly removes authority.

2.3 Layer 3: Treasury custody — Safe integration

Treasury custody is delegated to Safe {Wallet} multisig (safe.global) rather than implemented in the stack itself. The governance module is configured as a Zodiac module on the Safe, enabling the governor to execute approved proposals directly against the Safe's balance. This pattern provides several benefits:

- **Off-the-shelf custody UX.** The Safe Wallet web application provides treasury visibility, manual override capability, and broad ecosystem integration.
- **Multisig backstop.** The Safe's primary signer set serves as a manual fallback if the governance system fails or is compromised.
- **Established audit trail.** Safe's deployment history and audit posture are widely understood by counterparties.
- **Composability with other Zodiac modules.** Reality, Delay, Roles, Bridge, and other Zodiac modules can be added to the same Safe alongside the Lux Governor.

The trade-off is that the Safe team controls the custody contract evolution; mitigation is to deploy known-good versions and pin them.

2.4 Layer 4: Freeze and resume — circuit breaker

The freeze layer at `contracts/contracts/deployables/freeze-voting/` and `contracts/contracts/deployabl` implements an emergency-stop pattern.

Contract	Role
`FreezeGuardGovernorV1`	A Safe Guard hook that vetoes transaction execution while the DAO is in a frozen state. Mounted on the Safe alongside the governance module.
`FreezeVotingGovernorV1`	The vote-counting contract for freeze proposals (typically token-weighted).
`FreezeVotingMultisigV1`	Alternative freeze-voting contract for organizations using a multisig signer set rather than token holders.
`FreezeVotingStandaloneV1`	Standalone freeze-voting for SubDAO scenarios where the parent organization holds freeze authority.

The freeze mechanism is the corporate-governance equivalent of a shareholder injunction or a regulatory cease-and-desist. When freeze is voted in, the Safe Guard blocks all transactions until the freeze is lifted (by a separate vote, or by timeout). For a corporate adopter this is the cryptographic enforcement of an emergency board action: even if rogue authorities attempt to drain the treasury, the guard refuses the transaction. For a community fund, freeze provides protection against governance attack.

The Standalone variant is particularly important for hierarchical structures: a parent DAO can freeze a child without the child's cooperation, enabling on-chain corporate-parent oversight of subsidiaries.

2.5 Layer 5: Account abstraction and gas sponsorship

The account-abstraction layer at `contracts/contracts/deployables/account-abstraction/` integrates ERC-4337 account abstraction with the governance system.

Contract	Role
`PaymasterV1`	ERC-4337 paymaster that sponsors gas for designated users or transactions. Permits the organization to pay gas fees on behalf of members, employees, or eligible community participants.
`BasePaymaster`	Base paymaster providing common policy hooks.
`LightAccountValidator`	Account-validation logic for Alchemy LightAccount-pattern smart accounts. Enables gasless user accounts whose validation can be policy-controlled by the organization.

For corporate use, this layer enables an essential UX property: employees do not need to hold ETH to perform on-chain duties. The organization sponsors gas via the paymaster for transactions matching policy (for example, proposal-related transactions, hat-related transactions, or transactions touching specific approved contracts). Combined with LightAccount, employees use email-and-password (or single-sign-on) login flows backed by smart-contract accounts.

For community-fund use, the paymaster enables wider participation: voters do not need ETH balances to vote, and proposal authors do not need ETH to file proposals.

2.6 Layer 6: Token issuance and warrant tracking

The token-issuance layer combines voting-extended ERC-20 tokens with warrant-tracking and public-sale infrastructure.

Contract	Role
`VotesERC20V1`	Voting-extended ERC-20 token (delegate semantics from OpenZeppelin ERC20Votes). The canonical governance token for community-fund and shareholder-style governance.
`WarrantBase`	Base contract for warrant tracking. Warrants represent rights to acquire underlying tokens at a strike price under defined conditions.
`WarrantHedgeyV1`	Integration with Hedgey Finance for vesting and lockup plans. Standard pattern for employee or community-contributor token grants with cliff and vesting schedules.
`PublicSaleV1`	Public-sale contract for token launches. Handles allowlist enforcement, per-buyer caps, soft-cap and hard-cap mechanics, and refund handling.

For corporate use, this layer supports the equivalent of stock-option grants (Warrant + Hedgey lockup), founder share allocations (VotesERC20 with delegate semantics for board representation), and capital raises (PublicSale for token-equity-style fundraises).

For community-fund use, this layer supports the canonical community-fund launch pattern: VotesERC20 mint, vested community-allocations via WarrantHedgey, and an open token sale via PublicSale to bootstrap treasury and distribute governance authority.

2.7 Layer 7: Token-bound accounts and infrastructure

The infrastructure layer integrates ERC-6551 token-bound accounts and on-chain key-value storage.

Contract	Role
`ERC6551Registry`	Standard registry for token-bound accounts. Every NFT can address a smart-contract account that the NFT controls.
`KeyValuePairsV1`	On-chain string key-value storage. Stores DAO metadata (name, description, social links), proposal metadata, and configuration that does not require dedicated contract state.
`SystemDeployerV1`	Orchestrates deployment of the full DAO stack from a single transaction. Replaces multi-step deployment scripts with an atomic deployment ceremony.
`AutonomousAdminV1`	Admin-pattern contract for autonomous functions (timed actions, automatic execution of approved proposals).

ERC-6551 token-bound accounts are conceptually significant: every NFT in the system gets its own on-chain wallet. For a corporation, this means departments, subsidiaries, projects, and ventures can each be represented as an NFT whose token-bound account is the department's wallet. Ownership of the NFT (which can itself be held by a multisig or another smart account) determines authority over the wallet.

For a community fund, ERC-6551 enables "project NFT" models: the fund mints an NFT per funded project, the NFT's token-bound account receives the project's allocation, and the fund retains the NFT until the project is complete or terminated.

3 Deployment patterns

The seven layers combine to support several distinct deployment patterns. Each pattern is a coherent organizational architecture; the stack is designed to support all of them without modification.

3.1 Pattern A: Corporate treasury with role-based governance

A traditional corporation adopts on-chain treasury custody for some or all corporate funds. The deployment is:

- Safe multisig holding the treasury, with primary signers being executive officers.
- ModuleGovernorV1 attached as a Zodiac module, configured with StrategyV1.

- Hats Protocol role hierarchy: CEO, CFO, Treasurer, Board, Compliance Officer, etc. each as on-chain hats.
- ProposerAdapterHatsV1 routes proposals: only the Treasurer hat-holder may propose treasury motions; only the Board may propose governance changes.
- FreezeGuardGovernorV1 mounted on the Safe; the Compliance Officer hat can initiate a freeze pending Board review.
- PaymasterV1 sponsors gas for transactions authorized by hat-holders.
- KeyValuePairsV1 stores corporate metadata: legal name, jurisdiction, fiscal year, regulator references.

Operations: the Treasurer proposes a payment; the Board votes per ModuleGovernorV1 strategy; on success, the Safe executes. All actions are on-chain, audit-trailed, and verifiable.

3.2 Pattern B: Corporate parent with subDAO operating companies

A holding company structure with multiple operating subsidiaries. The deployment uses ModuleFractalV1:

- Parent Safe + ParentGovernor at the holding-company level.
- Multiple child Safes + ChildGovernors, each for an operating subsidiary, attached as fractal children of the parent.
- Per-child autonomy boundary: routine operations are decided by the subsidiary's own governance; major actions (capital allocation above X, M&A, IP transfer) require parent ratification.
- Parent-side freeze authority over any child (FreezeVotingStandaloneV1 with the parent's vote counting as the standalone freeze vote).
- Per-child Hats hierarchy with cross-references to the parent Hats hierarchy.
- Token-bound accounts (ERC-6551) for departments within each subsidiary; the department NFT is held by the subsidiary's Safe.

This is the on-chain analog to a typical multinational holding-company structure: parent retains strategic control, subsidiaries operate independently within budget, and there is a clear escalation path.

3.3 Pattern C: Token-launched community fund

A community-governed pool of capital. The deployment is:

- Mint VotesERC20V1 with a defined total supply.
- Distribute via WarrantHedgeyV1 (vested grants to core contributors) and PublicSaleV1 (open sale to bootstrap treasury and distribute governance).
- Safe multisig with the public-sale proceeds as the fund treasury.

- ModuleGovernorV1 + VotingWeightERC20V1 + StrategyV1 with low quorum and short voting period optimized for community throughput.
- Optional Hats Protocol: a Curator hat with proposer-gating to filter spam.
- PaymasterV1 sponsors voting and proposal transactions to maximize participation.
- ERC-6551 NFTs representing each funded project; the fund holds each NFT and transfers it (with locked vesting) when a project is funded.

Operations: anyone holding the governance token can vote; proposers must hold the Curator hat or meet a token threshold; approved proposals release funds from the Safe to project NFT accounts.

3.4 Pattern D: Hybrid corporate venture fund

A corporation operating a venture investment fund as a community-governed subDAO. Combines Pattern B (parent + subDAO) with Pattern C (community-governed fund):

- Parent corporation governance per Pattern A.
- Venture fund as a fractal subDAO under the parent.
- Community participation via a venture-fund-specific VotesERC20 distributed to limited partners (LPs) and key contributors.
- Investment proposals routed through ProposerAdapterHatsV1 with a General Partner hat held by the fund manager.
- Treasury custody on a fund-specific Safe with the parent retaining freeze authority.
- Per-investment NFTs (ERC-6551) representing portfolio company holdings; ownership controllable as a transferable asset.

This pattern is one of the most commercially interesting because it bridges traditional fund structures (GP/LP, fund commitments, investment committee approvals) with community-governance UX (transparent voting, on-chain audit trail, programmable governance).

3.5 Pattern E: Open community treasury with multi-jurisdictional reach

A community treasury operating across multiple legal jurisdictions, with per-jurisdiction governance branches. Combines fractal subDAOs with jurisdictional compartmentalization:

- Parent "Foundation" Safe + Governor at a Swiss / Cayman / DIFC / Singapore foundation.
- Per-jurisdiction operating Safes for activities in regulated markets (US, EU, UK), each with jurisdiction-specific compliance hats and freeze authority.
- Cross-jurisdictional treasury rebalancing via parent-Governor proposals.
- Per-jurisdiction-specific PublicSale / Warrant / Hats configurations matching local securities law.

4 Gaps and proposed extensions

The stack is functionally complete for unregulated and lightly-regulated use cases (Patterns C, D, E above). For regulated corporate use cases (Patterns A and B with traditional securities-law obligations) several gaps remain. Below we identify them and propose extensions tied to the Lux patent family.

4.1 Gap G1: Off-chain identity binding

The stack permits any address to hold a governance token, a hat, or a Safe signer position. Regulated corporate use cases require the identity holding each authority to be verified (KYB for entities, KYC for natural persons), and the on-chain identifier to be cryptographically bound to the off-chain identity. The stack does not currently provide this binding.

Proposed extension. Integrate ERC-3643 + OnchainID (per Lux Industries' `identity-bound-transfer-restr` provisional and Liquidity's existing ERC-3643 implementation). Every Safe signer, hat-holder, and token-holder address is bound to an OnchainID identity. Each identity carries claims (KYC, KYB, accreditation, jurisdiction) signed by trusted issuers (KYB providers, broker-dealers, transfer agents). Proposal execution conditions can reference identity claims: "execute only if proposer.identity has KYB-verified-corporate-officer claim from issuer X."

4.2 Gap G2: Regulated-securities cap-table integration

VotesERC20V1 supports community-fund use cases but does not satisfy SEC-registered transfer-agent obligations for corporate equity tokens. Specifically: Rule 144 holding-period enforcement, Regulation D/S/CF/A resale restrictions, and corporate-action processing (dividends, stock splits, redemptions) are not implemented.

Proposed extension. Integrate the Liquidity transfer-agent stack (`liquidityio/ta`) as the trusted issuer of claims that gate VotesERC20 transfers. Add a ResaleQuadrant compliance module: each transfer is gated by checking holder claims against the applicable exemption (Reg D, Reg S, Reg CF, Reg A+) and the holder's holding-period status. Combine with the PQ-threshold-MPC custody patent (provisional 2026-006) to provide cryptographically-enforced transfer-agent authority.

4.3 Gap G3: Post-quantum signature support

All current signature paths use classical primitives (ECDSA secp256k1 on Safe signers, BLS or ECDSA on token signing). For long-lived corporate treasury obligations (warrants vested over 4-10 years, corporate bonds with multi-decade maturities, sovereign-wealth-fund holdings) the lifetime of the relevant cryptographic operations may exceed the practical security horizon of classical signatures.

Proposed extension. Adopt the PQ master CIP (Lux original-filing 2026-001) and the crypto-agility framework (2026-005). Specifically: deploy a hybrid ECDSA+ML-DSA signature scheme for Safe signer authority, with epoch-based rotation to pure ML-DSA at a configurable migration epoch. Implement the parent-chain post-quantum verification primitives as precompiled contracts (per the Lux PQ consensus filing, 2026-007). Light-client verification matrix exposed to off-chain audit tooling.

4.4 Gap G4: Indexer and off-chain query layer

`STACK_REALITY.md` notes that the indexer service is unimplemented and the API backend exists but is not running. For corporate audit and reporting, off-chain indexing is required to produce account statements, voting-records, treasury movements, and regulatory reports without requiring auditors to scan the chain.

Proposed extension. Implement the indexer service against the existing PostgreSQL schema. Use the Liquid Tasks / Liquid Insights tooling already deployed in the Liquidity stack. Connect the API backend (already coded at `api/packages/lux-offchain`) to the frontend, enabling fast queries without RPC round-trips. Deploy the subgraph (also already partially implemented) for GraphQL-style historical queries. This is engineering work, not architectural work: the design exists, only execution remains.

4.5 Gap G5: Multi-jurisdiction compliance routing

Pattern E (multi-jurisdictional treasury) requires per-jurisdiction compliance enforcement at the contract layer. Today this would have to be coded per-jurisdiction; there is no general framework.

Proposed extension. Add a JurisdictionRouter contract that maintains per-jurisdiction policy modules. Each policy module is itself a ModuleFractalV1 child that enforces jurisdiction-specific rules. Proposal execution against any Safe controlled by the system queries the JurisdictionRouter to determine which policy applies based on transaction destination and parties involved.

4.6 Gap G6: Audit trail with cryptographic provenance

The current stack relies on Safe's transaction history and the blockchain's standard tx record. For SOC 2, SEC Rule 17a-4, and similar audit requirements, cryptographic-provenance audit trails with non-repudiation are required, including evidence of proper authorization at the time of each transaction.

Proposed extension. Add a CryptographicAuditLog contract that records, for each Safe-executed transaction, a tuple of (transaction hash, authorization chain identifying which proposal authorized this transaction, hat-holder signatures contributing to authorization, freeze-state at execution time). The contract emits an event consumable by off-chain audit tooling. Combine with the existing Liquid Insights audit infrastructure.

4.7 Gap G7: Treasury policy modules for institutional risk management

Treasury policy — maximum exposure to any single asset, maximum daily withdrawal, required diversification — is currently enforceable only through governance discipline. For corporate use cases requiring institutional risk management, contract-enforced policies are needed.

Proposed extension. Add a TreasuryPolicy contract that gates Safe transactions against configurable risk-policy parameters. Examples: maximum 10% of treasury value in any single asset; maximum 5% of treasury value withdrawn per 24-hour window; required minimum stablecoin balance. Mounted as a Safe Guard alongside the FreezeGuard.

5 Strategic positioning

The Lux DAO stack occupies an unusual position relative to competing on-chain governance systems. Competitors include:

- **Aragon (osx).** Modular governance framework. Strong on plugin architecture. Less integrated with role hierarchies; no built-in cap-table or warrant support.
- **Tally / Compound Governor.** Reference OpenZeppelin Governor implementations. Production-grade but minimal scope: governance only, no role hierarchy, no freeze, no AA, no warrant.
- **Snapshot.** Off-chain voting with on-chain execution via Safe. Wide adoption but no contract-enforced governance — requires social-layer trust in execution.
- **Moloch / Daohaus.** Veteran on-chain DAO framework. Mature but limited extension; no Hats integration, no AA.
- **Decent (Fractal — the upstream of Lux DAO).** Direct upstream. Lux DAO inherits design but extends with the Lux Governor branding and integration with the broader Lux ecosystem.

The Lux DAO stack's competitive position is:

- More feature-complete than reference Governors (freeze, AA paymaster, warrant, public sale, ERC-6551 all included).
- More integrated with traditional corporate structures than community-fund-oriented frameworks (Moloch, Daohaus).
- More extensible than monolithic systems (Aragon plugins, Snapshot off-chain).
- Most importantly: **the only governance stack tied to a regulated-securities cap-table layer (Liquidity TA) and a post-quantum patent family**.

  Combined with the Lux PQ patent portfolio (provisionals 2026-001 through 2026-007), the Lux DAO stack becomes the *regulated-corporate-governance moat* for the institutional market: any institution wanting to operate on-chain governance with cap-table integrity, transfer-agent enforcement, and forward-secure cryptographic provenance is structurally directed to the Lux substrate.

6 Roadmap to production-grade regulated use

The gap analysis above suggests a focused 90-day engineering plan that brings the stack to production-grade for regulated corporate use cases.

Sprint 1 (weeks 1-4): Indexer + API activation.

- Implement the indexer service against the existing schema.
- Wire the API backend (already coded) into the Docker compose and production manifests.
- Connect frontend to the API for historical queries; retain direct RPC for live transactions.

- Deploy subgraph for additional GraphQL query patterns.
- Closes Gap G4.

Sprint 2 (weeks 5-8): Identity + cap-table.

- Integrate ERC-3643 + OnchainID for identity binding.
- Wire VotesERC20V1 transfer hooks to query the OnchainID for KYC / accreditation / jurisdiction claims.
- Integrate with the Liquidity TA (`liquidityio/ta`) as a trusted-issuer of compliance claims.
- Add ResaleQuadrant compliance module enforcing Rule 144 / Reg D / Reg S / Reg CF / Reg A+ resale restrictions.
- Closes Gaps G1 and G2.

Sprint 3 (weeks 9-12): PQ migration + audit trail + treasury policy.

- Add hybrid ECDSA+ML-DSA support to Safe signer authority, gated by the agility policy contract.
- Add CryptographicAuditLog contract and event emission.
- Add TreasuryPolicy guard contract for institutional risk management.
- Closes Gaps G3, G6, G7.

After this 90-day program, the stack is production-grade for Patterns A through D at institutional scale. Pattern E (multi-jurisdiction) requires Gap G5's JurisdictionRouter, estimated at an additional 4-6 weeks.

7 Conclusion

The Lux DAO stack is one of the most architecturally complete on-chain governance frameworks available, combining (a) modular Zodiac-style governance, (b) Hats-based role hierarchies for corporate-style delegation, (c) Safe-backed treasury custody with circuit-breaker freeze, (d) ERC-4337 account abstraction with sponsoring paymasters for participation UX, (e) warrant and public-sale machinery for token-equity and capital-raise mechanics, and (f) ERC-6551 token-bound accounts for hierarchical asset organization. For community-fund and lightly-regulated corporate use cases the stack is production-ready today.

For regulated corporate use cases the stack requires extensions in seven specific areas, each of which is well-scoped and tied to existing Lux ecosystem components (the Liquidity TA, the ERC-3643 identity layer, the PQ patent family). A 90-day focused engineering program closes the gaps necessary for the principal regulated use cases.

The strategic position of the stack is unique: it is the only on-chain governance framework with a coherent path to integration with regulated-securities cap-table management, post-quantum cryptographic primitives, and an enforcement-ready patent portfolio covering rollup substrate, cross-chain

messaging, and institutional tokenization. Closing the identified gaps converts the stack from an "open-source DAO framework" into the institutional-governance moat for the Lux ecosystem.

Cross-references

- Lux DAO repository: `~/work/lux/dao/`
- Liquidity transfer-agent service: `~/work/liquidity/ta/`
- ERC-3643 identity stack: `~/work/liquidity/contracts/lib/standard/contracts/identity/`
- Patent overview: `~/work/lux/patents/acquired/us-11803853-nahmii-rollup-control/`
- PQ master CIP application: `~/work/lux/patents/original-filings/2026-001-pq-transaction-contro`
- PQ regulated-custody application: `~/work/lux/patents/original-filings/2026-006-pq-threshold-mpc`
- PQ institutional-settlement application: `~/work/lux/patents/original-filings/2026-004-pq-institut`
- Crypto-agility framework: `~/work/lux/patents/original-filings/2026-005-crypto-agile-validatio`